ChatGPT

canirunai SDK 설계 문서

작성일: 2026-03-18 (Asia/Seoul)

본 문서는 가칭 **canirunai**(Python 기반 SDK)로, 주어진 CPU/GPU/RAM 및 AI 모델 스펙(주로 LLM)을 기반으로 **해당 하드웨어에서 해당 모델 추론 서버를 얼마나 "잘" 실행할 수 있는지**를 정량(0~100 점수) 및 정성(verdict) 지표로 산정해 JSON으로 제공하는 SDK의 전반 설계를 정의한다.

SDK는 추후 웹 서비스로 확장될 것을 전제로 하되, **SDK(코어)와 CLI/터미널 UI는 완전히 디커플링**한다.

제품 개요와 네이밍

목표와 핵심 출력

canirunai의 핵심 목표는 다음 입력이 주어졌을 때:

- CPU 스펙(복수 CPU 지원)
- GPU 스펙(복수 GPU 지원)
- RAM 용량(사용자 입력을 single source of truth로 사용)
- AI 모델 스펙(주로 Hugging Face 기반 LLM)

다음 결과(최소 필드)를 산출하는 것이다.

- `verdict`: `RUNS GREAT` / `RUNS WELL` / `TIGHT FIT` / `TOO HEAVY` / `IMPOSSIBLE`
- `score`: 0~100 정수
- `context_estimate`: (`safe_context_tokens`, `max_supported_context_tokens`)
- `throughput_estimate`: (`decode_tokens_per_sec`, `prefill_tokens_per_sec`)

추가로 `--output wide`에서 제공 가능한 "가능한 모든 정보"에는 VRAM/RAM 점유 추정, 병목 요약(원인만), 배치/복제 배치 추정(placement), 토큰 생성 지연 시간(정규화된 형태) 등을 포함한다.

네이밍 제안

- canirunai는 "이 하드웨어로 AI를 돌릴 수 있나?"라는 의미 전달이 직관적이므로 유지 가능하다(가제).
- 대안 제안(가독성/발음/브랜딩 관점):
- **inferfit**: inference + fit, 기능을 직접적으로 표현
- **runfitai**: 실행 적합도 평가를 강조
- **modelhostfit**: 모델-호스트 적합도에 초점
- **can-i-run-llm**: 기능 명확(다만 패키지 네이밍엔 다소 길 수 있음)

최종 선택은 패키지명(Python import 경로)과 CLI 명령어(`canirunai`)를 분리할 수 있도록 설계한다(예: 패키지 `inferfit`, CLI `canirunai`도 가능).

데이터 소스와 수집 범위

본 SDK의 “스펙 수집기”는 웹 크롤러 성격이며, 사이트별 표현이 상이한 비정형(또는 반정형) HTML을 SDK 스키마로 정형화한다. 특히 Wikipedia, Hugging Face는 HTML/UI가 바뀔 수 있으므로, **raw HTML 저장 + LLM 기반 파싱 + 스키마 검증**으로 견고성을 확보한다.

CPU 스펙 소스와 수집 가능한 필드

CPU는 Wikipedia의 다음 페이지를 사용한다(요구사항 원문 그대로 반영).

- 인텔 CPU: “List of Intel processors”
- AMD Ryzen CPU: “List of AMD Ryzen processors”

이들 페이지는 섹션/세대별 표 구성이 다르지만, 공통적으로 모델별로 코어/스레드/클럭/캐시/TDP/소켓/출시 시점 같은 추론 적합도 산정에 유용한 필드를 제공한다.

예를 들어 인텔 목록에는(세대/라인업별 테이블에서) `Model`, `Cores`, `Threads`, `Clock rate (GHz)` (Base/Max turbo), `Cache`, `IGP`, `TDP`, `Codename`, `Socket`, `Release` 같은 열이 tabel header 수준에서 반복적으로 나타난다. [1]

AMD Ryzen 목록은 데스크톱/모바일/임베디드 등으로 나뉘며, 테이블에 `Cores (threads)`, `Clock rate (GHz)`(Base/Boost), `L3 cache`, `TDP`, `Release date` 등이 포함되고, 일부 구간(특히 모바일/임베디드 표)은 `Fab`, `Socket`, `PCIe support`, `Memory support`, 캐시(L1/L2/L3)까지 더 풍부한 스펙을 제공한다. [2]

따라서 canirunai의 CPU 스키마는 “항상 존재하는 최소 필드” + “존재할 수 있는 확장 필드”를 함께 수용해야 한다.

GPU 스펙 소스와 수집 가능한 필드

GPU는 Wikipedia의 다음 페이지를 사용한다(요구사항 원문 그대로 반영).

- NVIDIA GPU: “List of Nvidia graphics processing units”
- AMD GPU: “List of AMD graphics processing units”

NVIDIA 목록은 모델별로 `Launch Date`, `Code Name`, `Fab (nm)`, `Transistors`, `Bus Interface`, `Core Clock`, `Memory Clock`, `Memory Size`, `Bandwidth (GB/s)`, `Bus Type`, `Bus Width`, `Fillrate`, `TDP` 등 LLM 추론의 메모리/전력 특성에 직접 연결되는 항목을 포함하는 테이블을 제공한다. 또한 일부 테이블은 `CUDA Compute Capability` 및 `Processing power (GFLOPS)` 같은 계산/호환성 단서를 추가로 제공한다. [3]

AMD GPU 목록도 유사하게 메모리/대역폭/TDP/API 호환성 및 일부 구간의 연산 성능(GFLOPS) 정보를 포함하는 표 구조를 제공한다. 또한 “세대별 관례가 달라 수치들이 1:1 비교에 부적절할 수 있음”을 경고하고 있어, SDK 점수 산정에서 **정확한 벤치마크가 아닌 근사 모델**임을 시스템적으로 명시할 필요가 있다. [4]

모델 스펙 소스와 수집 가능한 필드

모델 스펙은 **Hugging Face Hub**를 대상으로 하며, 하위 페이지를 순회(또는 공식 API를 통해 동일 정보를 수집)한다. Hugging Face에서 “모델 카드(Model Card)”는 모델 리포지토리의 `README.md`가 렌더링되는 것이며, 상단 YAML 메타데이터가 검색/발견성에 활용된다. [5]

라이선스는 Hugging Face에서 리포지토리 카드(README) 메타데이터로 지정 가능하며, Apache 2.0 라이선스 식별자는 `apache-2.0`이다. [6]

Hugging Face 모델 목록 페이지 UI는 Task, 파라미터 규모, 라이브러리, 앱 생태계(vLLM/llama.cpp/Ollama 등), Inference Providers, "Inference Available" 같은 필터를 제공하므로, 수집 범위 제한(LLM/Apache 2.0/Inference 가능/특정 조직) 정책을 구현하는 단서로 활용할 수 있다. [7]

특히 Inference Providers 관점에서 Hub는 `/api/models` 필터를 지원하며, `inference_provider` 파라미터로 "특정 provider가 서빙하는 모델" 혹은 `inference_provider=all`로 "어떤 provider든 서빙하는 모델"을 나열할 수 있고, `model_info(..., expand="inference")`로 `warm` 여부(또는 None)를 확인할 수 있다. [8]

또한 Hugging Face의 Python 클라이언트(`huggingface_hub`) 문서상 `list_models()`는 `pipeline_tag`, `num_parameters` 범위, `author`, `inference`, `inference_provider` 등 필터를 지원하며, `expand`로 `config`, `safetensors`, `gguf`, `siblings`, `inferenceProviderMapping` 같은 확장 속성을 선택적으로 가져올 수 있다. [9]

모델 수집 범위의 현실적 제안

"Hugging Face의 모든 모델"은 규모가 지나치게 크므로(모델 페이지 자체가 수백만 단위를 표기), canirunai는 기본 수집 정책을 강하게 제한해야 한다. [10]

기본 정책(설계 제안):

- Task: `pipeline_tag=text-generation` (LLM 중심)
- License: `apache-2.0` (요구사항)
- Inference 가능: `inference_provider=all` 또는 `expand="inference"`에서 `warm`인 모델(요구사항의 "hf에서 inference 기능 지원 가능"을 시스템적으로 해석)
- Author(Organization/User): config에 정의된 allowlist
  (예: Qwen, openai, meta-llama, google, deepseek-ai, MiniMaxAI, mistralai, lgai-exaone, nvidia 등)
- 규모 제한: 조직별 상위 N개(예: downloads/likes/last_modified 기준) + 전체 상한(예: 2,000개)
- 파라미터 범위 기본값: `"min:0.3B,max:200B"` (config로 변경 가능)
  Hugging Face 검색은 UI와 동일한 range syntax를 지원한다. [11]

양자화 variant 취급(요구사항 반영):

- 동일 base 모델의 양자화 variant는 **별개 모델**로 저장한다.
- 다만 "한 리포지토리 안에 수십 개 GGUF 파일(수십 variant)"이 있을 수 있어 폭발을 막기 위해 기본 정책을 둔다.
- 예: GGUF는 Q4/Q5/Q8 대표 3종만 샘플링(파일명 패턴 기반)
- GPTQ/AWQ/NF4 등은 리포지토리 단위 별도 모델로 취급(대부분 별도 repo로 분리되어 있음)
- 이 로직은 추후 **Ollama** 등으로 확장 가능하도록 "Variant Extractor"를 인터페이스로 분리한다.

스펙 JSON 스키마 정의

스펙은 JSON 포맷으로 관리하며, 각 엔트리(예: CPU 1개, GPU 1개, Model 1개)는 반드시 다음 메타데이터를 포함한다.

- `collected_at` (ISO 8601)
- `source_url` (원본 페이지)

- 가능하면 `source_revision` 또는 `source_sha`
- Wikipedia는 revision id를 수집하는 것이 바람직(페이지 변경 추적)
- Hugging Face는 repo `sha`, `last_modified` 등도 함께 저장 가능 [12]

저장 파일 구조 제안

- `data/specs/cpu.json`
- `data/specs/gpu.json`
- `data/specs/model.json`
- `data/raw_cache/...` (수집 당시 raw HTML/요약 텍스트/LLM 입력 chunk를 저장)

각 spec file은 "컬렉션 단위 메타"와 "items"로 구성한다.

```
{
 "schema_version": 1,
 "generated_at": "2026-03-18T12:00:00+09:00",
 "source": {
  "collector": "canirunai.cpu_wikipedia",
  "notes": "merged incremental update"
 },
 "items": [
  { "... one item ..." }
 ]
}
```

CPU 스펙 스키마 (Pydantic 기준)

CPU는 표 구성이 섹션마다 달라 누락 필드가 흔하므로, 대부분 `Optional`로 설계한다.

```
{
 "kind": "cpu",
 "canonical_name": "Intel Core i7-8086K",
 "vendor": "intel",
 "family": "Core i7",
 "model": "8086K",
 "cores": 6,
 "threads": 12,
 "base_clock_ghz": 4.0,
 "boost_clock_ghz": 5.0,
 "l3_cache_mb": 12.0,
 "tdp_w": 95,
 "codename": "Coffee Lake",
 "socket": "LGA 1151",
 "release": "Q2 2018",
 "collected_at": "2026-03-18T12:00:00+09:00",
 "source_url": "https://en.wikipedia.org/wiki/List_of_Intel_processors",
 "source_revision_id": 123456789,
 "raw_ref": {
  "cache_key": "raw/wiki/intel_processors/section_x/table_y.html"
```

```
  }
}
```

이 스키마는 인텔 목록 테이블이 실제로 `Model`, `Cores`, `Threads`, `Clock rate`, `Cache`, `IGP`, `TDP`, `Codename`, `Socket`, `Release` 구조를 제공한다는 사실에 기반해 필드명을 정한다. [1]

GPU 스펙 스키마 (Pydantic 기준)

GPU는 LLM 추론 적합도에서 **VRAM 용량**과 **메모리 대역폭(GB/s)**의 영향이 크므로(정확한 벤치마크는 아니더라도), Wikipedia가 제공하는 Bandwidth/Memory Size/TDP를 최우선 수집 대상으로 둔다. [13]

```
{
  "kind": "gpu",
  "canonical_name": "NVIDIA GeForce RTX 4090",
  "vendor": "nvidia",
  "product_line": "GeForce",
  "architecture": "Ada Lovelace",
  "codename": "AD102",
  "process_nm": 4,
  "bus_interface": "PCIe 4.0 x16",
  "memory_size_gib": 24,
  "memory_bus_width_bit": 384,
  "memory_bandwidth_gbs": 1008.0,
  "tdp_w": 450,
  "api_support": {
    "vulkan": "1.x",
    "direct3d": "12",
    "opengl": "4.x"
  },
  "cuda_compute_capability": "8.9",
  "processing_power_fp32_gflops": 82600.0,
  "collected_at": "2026-03-18T12:00:00+09:00",
  "source_url": "https://en.wikipedia.org/wiki/List_of_Nvidia_graphics_processing_units",
  "source_revision_id": 987654321,
  "raw_ref": {
    "cache_key": "raw/wiki/nvidia_gpus/desktop/table_geforce_x.html"
  }
}
```

NVIDIA 목록에는 “Memory Size/Bandwidth/Bus Type/Bus Width/TDP/API” 및 일부 구간에서 “CUDA Compute Capability/Processing power(GFLOPS)” 같은 항목이 테이블 헤더로 제공된다. [3]

AMD GPU 목록 역시 메모리/대역폭/TDP/API/연산 성능(GFLOPS) 등의 항목을 테이블로 제공하고, 세대 간 수치 비교에 주의하라는 경고를 포함한다. [4]

모델 스펙 스키마 (Pydantic 기준)

모델 스펙은 “카드(README) + config + 파일 목록(사이즈)”을 수집해 **가능한 한 자동으로 파라미터 수/컨텍스트 길이/아키텍처 단서**를 도출하는 방향을 택한다.

Model card는 README.md이며 YAML 메타데이터를 포함할 수 있다. [14]

Hugging Face `ModelInfo`는 `config`, `downloads`, `downloads_all_time`, `card_data`, `siblings`(파일 목록) 등을 제공한다. [15]

```
{
  "kind": "model",
  "canonical_name": "deepseek-ai/DeepSeek-R1@bf16",
  "source": "huggingface",
  "hf_repo_id": "deepseek-ai/DeepSeek-R1",
  "variant": {
    "precision": "bf16",
    "quantization": null,
    "format": "safetensors"
  },
  "task": "text-generation",
  "license_id": "apache-2.0",
  "inference": {
    "provider_required": true,
    "status": "warm"
  },
  "declared_context_tokens": 32768,
  "architecture_hint": {
    "model_type": "llama",
    "num_layers": 80,
    "hidden_size": 8192,
    "num_attention_heads": 64,
    "num_kv_heads": 8,
    "vocab_size": 128000
  },
  "num_parameters": 70000000000,
  "weights": {
    "total_size_bytes": 140000000000,
    "files": [
      {
        "path": "model-00001-of-000xx.safetensors",
        "size_bytes": 10000000000
      }
    ]
  },
  "popularity": {
    "downloads_30d": 123456,
    "likes": 7890
  },
  "collected_at": "2026-03-18T12:00:00+09:00",
  "source_url": "https://huggingface.co/deepseek-ai/DeepSeek-R1",
  "source_sha": "abcdef123456"
}
```

수집 구현은 `huggingface_hub`의 `list_models()`/`model_info()` 기반을 기본으로 권장한다. 해당 API는 Hub의 OpenAPI 명세로 관리되며, Hub API 문서는 OpenAPI 스펙(.well-known/openapi.json)을 제공한다고 밝힌다. [16]

또한 `list_models()`는 `pipeline_tag`, `num_parameters` 범위, `author`, `inference_provider`, `inference` 등을 직접 인자로 제공한다. [17]

패키지 아키텍처와 모듈 설계

고수준 모듈 구성

요구사항을 반영해 canirunai는 다음 모듈 집합으로 구성한다("etc" 포함 확장 가능).

- 스펙 수집기(크롤러)
- `collectors.cpu_wikipedia`
- `collectors.gpu_wikipedia`
- `collectors.model_huggingface`
- OpenAI API 기반 파서 모듈(LLM 정형화)
- `parsers.openai_structured_parser`
- Pydantic 스펙 정의 모듈
- `schemas.cpu`, `schemas.gpu`, `schemas.model`, `schemas.score`
- 스펙 로더/스토어
- `store.json_store`
- `loaders.cpu_loader`, `loaders.gpu_loader`, `loaders.model_loader`
- 점수 산정 모듈
- `scoring.engine`
- `scoring.llm_estimator` (LLM 전용)
- CLI(Click 기반) 파서 모듈
- `cli.main` (단, 코어와 완전 디커플링)
- 터미널 출력 모듈(pretty print)
- `ui.terminal_printer`
- 로깅
- 전 모듈 공통: Loguru

디렉터리 구조(권장)

```
canirunai/
  __init__.py
  config/
    loader.py
    default_config.toml
  schemas/
    base.py
    cpu.py
    gpu.py
    model.py
    score.py
  collectors/
```

```
    base.py
    wikipedia_client.py
    cpu_wikipedia.py
    gpu_wikipedia.py
    huggingface_client.py
    model_huggingface.py
  parsers/
    openai_structured_parser.py
    normalization.py
  store/
    json_store.py
    raw_cache.py
  loaders/
    cpu_loader.py
    gpu_loader.py
    model_loader.py
  scoring/
    engine.py
    llm_estimator.py
    verdict.py
  ui/
    terminal_printer.py
  cli/
    main.py
    resources/
      update.py
      list.py
      get.py
      check.py
  tests/
   ...
```

Wikipedia 수집 클라이언트 설계

요구사항은 "웹사이트 크롤링"이지만, Wikipedia는 직접 HTML 스크래핑보다 **MediaWiki API를 통한 페이지/테이블 HTML 확보**가 더 안정적이고 변경 추적(revision id)도 쉬워 권장된다.

- `action=parse`: 페이지를 파싱해 HTML을 얻는 방식("parse content of a page and obtain the output")으로 문서화되어 있다. [18]
- `action=query&prop=revisions`: revision 정보를 얻는 API가 공식 문서로 제공된다. [19]
- 페이지 content 획득은 Revisions API로 wikitext를 받을 수 있으며 `rvprop=content`를 사용한다. [20]

이 설계는 결과 JSON에 `source_revision_id`를 저장할 수 있게 하며, 변경분 업데이트(update)가 "diff 기반"으로도 확장 가능해진다(향후 최적화).

추가 고려: Wikimedia는 무분별한 봇 스크래핑이 인프라에 부담을 준다는 문제의식을 공개적으로 표명해 왔으므로, canirunai의 수집기는 기본적으로 강한 rate limit/throttle/cache 정책을 내장하는 방향이 바람직하다. [21]

OpenAI 기반 파서(정형화) 모듈 설계

Wikipedia/Hugging Face는 UI/테이블 표현이 바뀌거나 섹션별로 헤더가 달라 "순수 rule-based 파서"만으로 장기 운영이 어렵다. 따라서:

1. 수집기는 raw HTML(또는 table HTML chunk)을 확보한다.
2. LLM 파서가 raw HTML을 "스키마에 맞는 JSON"으로 변환한다.
3. Pydantic 검증 실패 시 자동 재시도(입력 축소/명확화/부분 파싱)한다.

OpenAI API는 JSON mode 및 Structured Outputs(스키마 강제)를 제공한다. JSON mode는 "유효한 JSON"을 보장하지만 스키마 일치를 보장하지 않으므로, 본 프로젝트 같은 "정형화 모델링"에는 Structured Outputs 중심 설계가 적합하다. [22]

또한 OpenAI는 새 통합 프리미티브로 Responses API를 권장하며, Structured Outputs/Function calling을 포함한 도구 호출 중심 활용을 공식 가이드로 제시한다. [23]

파서 모듈은 다음 요구사항을 만족해야 한다.

- 입력: `raw_html` + `source_url` + "해당 사이트의 테이블 맥락"(예: "Intel 8th gen table")
- 출력: 해당 스키마(`CpuSpec[]`, `GpuSpec[]`, `ModelSpec[]`)에 맞는 JSON
- 실패 시: 스키마 검증 오류를 프롬프트에 포함해 재시도(최대 N회)
- 로깅: loguru로 request_id, 모델, 토큰 사용량, 실패/재시도 사유 기록

점수 정보 산정 방식과 Verdict 정의

점수 산정은 "스펙 JSON 파일 + 사용자가 명시한 RAM 용량"만을 single source of truth로 사용한다(요구사항). 즉, 실행 시점의 실제 하드웨어 탐지, 벤치마크 실행, 드라이버/CUDA 감지 등은 core scoring에 포함하지 않는다.

전제와 산정 대상

- 모델은 기본적으로 LLM(text-generation)을 1차 대상으로 한다.
- 다중 GPU는 기본적으로 "모델 복제(replication)로 처리량 확장"을 우선 가정한다.
- 분산 텐서 병렬/파이프라인 병렬은 추후 확장 항목으로 별도 feature flag로 둔다.
- 다중 CPU는 "토크나이저/전처리 및 서버 오케스트레이션 처리 여력"으로만 반영하며, GPU 추론이 가능한 경우 GPU가 1차 병목이 되는 것으로 가정한다.

핵심 중간 산출물

LLM 점수 산정을 위해, 다음 중간 값을 산출한다.

- `weights_bytes`: 모델 가중치 바이트(variant 반영)
- 우선순위: (1) `num_parameters` ×bits/8, (2) Hugging Face 파일 목록(siblings) 합산
  `ModelInfo`가 `config`, `siblings`를 포함할 수 있다는 점을 기반으로 한다. [24]
- `kv_bytes_per_token`: KV cache 토큰당 바이트(추정)
- 모델 config에서 `num_hidden_layers`, `hidden_size`, `num_key_value_heads` 등을 사용
- `vram_required_at_context(C)`: 특정 컨텍스트 C에서 VRAM 요구량
- `weights + runtime_overhead + kv_bytes_per_token*C`
- `max_supported_context_tokens`: VRAM 한도 내 최대 컨텍스트
- `safe_context_tokens`: 안정 운용을 위한 안전 컨텍스트(예: max의 80~90% 및 모델 선언 컨텍스트 상한 내)

- `decode_tokens_per_sec`, `prefill_tokens_per_sec`: 처리량 추정치

VRAM 기반 컨텍스트 산정

각 GPU(멀티 GPU이면 "사용 GPU 집합")에 대해:

- `available_vram_bytes = memory_size_gib * 1024^3`
- `runtime_overhead_bytes = available_vram_bytes * overhead_ratio`
- 기본값 예: 0.08 (config로 제어)
- `max_context_tokens = floor((available_vram_bytes - weights_bytes - runtime_overhead_bytes) / kv_bytes_per_token)`

멀티 GPU 복제 모드에서는 **각 replica는 단일 GPU에 완전히 적재되어야** 하므로, effective VRAM은 "사용 GPU 중 최소 VRAM"으로 결정한다.

GPU 기반 토큰 처리량(throughput) 산정

Wikipedia GPU 목록은 메모리 대역폭(GB/s)과 일부 구간의 연산 성능(GFLOPS), CUDA compute capability 등을 제공한다. [13]

canirunai v1의 처리량 모델(설계 제안)은 다음 두 제한 중 더 작은 값을 취한다.

- **대역폭 제한(bandwidth-limited)**:
  `tps_bw = (memory_bandwidth_bytes_per_sec * eff_bw) / bytes_per_token_work`
- **연산 제한(compute-limited)**(가능한 GPU만):
  `tps_flops = (fp32_flops_per_sec * eff_flops) / flops_per_token_work`

여기서 `bytes_per_token_work`, `flops_per_token_work`는 간단한 근사로 시작한다.

- `flops_per_token_work ≈ 2 * num_parameters`
  (가중치 1개당 multiply-add 2 FLOPs 근사)
- `bytes_per_token_work ≈ weights_bytes * stream_reuse_factor`
  (decode 단계에서 weights streaming 성격을 근사)

`eff_bw`, `eff_flops`, `stream_reuse_factor`는 "정확한 벤치마크 수치"가 아니라 **스펙 기반 추정치**임을 명시하고, config에서 보정 가능하도록 한다. AMD GPU 목록이 세대간 수치 비교 주의(관례 변화)를 언급하므로, 특정 세대에 과최적화된 하드코딩 상수는 지양한다. [4]

멀티 GPU 복제 모드는 처리량을 단순 합산하지 않고, CPU 및 메모리 여력에 따른 상한을 둔다(예: `cpu_thread_cap_ratio`, `io_cap_ratio`). 이 CPU 상한은 "전처리/스케줄링" 관점의 안전장치이며, 구체 파라미터는 v1에서는 보수적으로 설정한다.

Verdict 규칙

Verdict는 "단일 점수"의 라벨링이 아니라, 반드시 **하드 게이트(IMPOSSIBLE) → 실사용성(TOO HEAVY/TIGHT FIT) → 양호(RUNS WELL/GREAT)** 순으로 판정한다.

- `IMPOSSIBLE`
- 어떤 GPU에서도 `weights_bytes + runtime_overhead_bytes`가 적재 불가
- 또는 `safe_context_tokens < min_context_tokens` (기본 2048, config)
- `TOO HEAVY`

- 적재는 가능하나 `safe_context_tokens`가 매우 작거나(예: <4096) `decode_tokens_per_sec`가 실사용 하한 미만(예: <5)인 경우
- `TIGHT FIT`
- `safe_context_tokens`는 확보되나 VRAM headroom이 작고(예: <10%) 동시성 여력이 낮은 경우
- `RUNS WELL`
- `safe_context_tokens` 충분 + decode 속도 양호 + headroom 및 동시성 여유 보통
- `RUNS GREAT`
- `safe_context_tokens` 매우 넉넉 + decode 속도 우수 + multi-GPU 복제 시 처리량 확장 여지 큼

최종 산출 JSON 스키마(ScoreReport)

점수 산정 모듈은 JSON을 return한다(요구사항). 기본 출력(기본 CLI)과 wide 출력 공통 기반 스키마는 다음처럼 정의한다.

```
{
 "schema_version": 1,
 "verdict": "RUNS WELL",
 "score": 78,
 "inputs": {
  "cpu": ["AMD Ryzen 9 7950X"],
  "gpu": ["NVIDIA GeForce RTX 4090"],
  "memory_gb": 64,
  "model": "deepseek-ai/DeepSeek-R1@bf16"
 },
 "placement_estimate": {
  "mode": "replicated_serving",
  "single_gpu_loadable": true,
  "replica_count": 1,
  "used_gpu_canonical_names": ["NVIDIA GeForce RTX 4090"]
 },
 "context_estimate": {
  "max_supported_context_tokens": 32768,
  "safe_context_tokens": 16384
 },
 "throughput_estimate": {
  "decode_tokens_per_sec": 42.0,
  "prefill_tokens_per_sec": 1200.0
 },
 "wide": {
  "memory_estimate": {
   "weights_vram_gb": 38.2,
   "runtime_overhead_vram_gb": 2.1,
   "kv_cache_gb_per_1k_tokens": 0.42,
   "total_vram_gb_at_safe_context": 47.1,
   "vram_headroom_gb": 0.9
  },
  "latency_estimate": {
   "first_token_ms_per_1k_prompt_tokens": 850,
   "generation_ms_per_128_output_tokens": 3050
  },
```

```
    "bottlenecks": {
      "primary": "gpu_vram",
      "secondary": "gpu_bandwidth"
    },
    "confidence": {
      "context": "high",
      "throughput": "medium"
    }
  }
}
```

CLI 설계와 디커플링 원칙

디커플링 원칙

- SDK 코어는 “import해서 사용하는 라이브러리”가 기본이다.
- CLI(`python -m canirunai.cli` 또는 `canirunai` 엔트리포인트)는 코어를 호출하는 얇은 래퍼이며, 코어 모듈은 CLI/터미널 렌더링에 의존하지 않는다.
- `terminal_printer`는 “JSON→pretty print”만 책임지고, 점수 산정 로직을 포함하지 않는다.

kubectl 스타일 커맨드 구조

요구사항에 “update resource에 cpu, gpu, mode”라고 쓰였으나, 이후 설명은 `model`을 사용하므로 본 문서는 `mode`를 `model`로 **정정**한다(의도상 명백한 오타). 또한 “gpu 업데이트에서 cpu 수집 기능 실행”도 타이포로 보고 문서에서 정정한다.

- command: `canirunai`
- subcommand: `update`, `list`, `get`, `check`
- resource: `cpu`, `gpu`, `model` (check는 resource/name 없음)
- flags: `--output {wide|json}` 등

update

- `canirunai update cpu`
- Wikipedia CPU 목록을 크롤링/갱신하고 `cpu.json` 업데이트
- CPU name을 canonical name으로 정규화(유니크 보장)
- `canirunai update gpu`
- Wikipedia GPU 목록을 크롤링/갱신하고 `gpu.json` 업데이트
- GPU name을 canonical name으로 정규화(유니크 보장)
- `canirunai update model [--hfname {hf repo id}]`
- Hugging Face 모델 스펙 수집/갱신 후 `model.json` 업데이트
- `--hfname`가 있으면 단일 repo 페이지만 수집
- 모델 기본형의 양자화 variant는 별개 모델로 취급

list

- `canirunai list cpu|gpu|model [--output wide|json]`
- 기본은 이름 목록
- `--output wide`는 스펙 요약 포함
- `--output json`은 pretty print 전 raw JSON 출력

get

- `canirunai get cpu {cpu name}`
- `canirunai get gpu {gpu name}`
- `canirunai get model {model name}`
- name에 해당하는 스펙 1건 상세 출력
- `--output json` 지원

check

- `canirunai check --cpu {cpu name} --gpu {gpu name} --memory {GB} --model {model name} [--output wide|json]`
- `--cpu`, `--gpu`는 여러 번 사용 가능(멀티 CPU/GPU)
- 기본 출력은 `verdict/score/context/tps`만 포함
- `--output wide`는 가능한 모든 정보를 포함
- `--output json`은 terminal_printer 이전의 raw JSON 출력

config.toml, 저장 정책, 레이트리밋/캐시

config.toml 원칙

- 모든 모듈 설정은 `config.toml`에 집중한다(요구사항).
- 민감 정보(OpenAI API Key)는 환경변수 override도 지원한다.
- 수집기(collector)마다 source URL, rate limit, 저장 경로를 분리한다.

예시:

```
[sdk]
data_dir = "./data"
raw_cache_dir = "./data/raw_cache"
log_level = "INFO"

[openai_parser]
api_key = "ENV:OPENAI_API_KEY"
base_url = "https://api.openai.com/v1"
model = "gpt-5"
max_retries = 3
timeout_sec = 60

[wikipedia]
user_agent = "canirunai/0.1 (contact: your-email)"
request_delay_sec = 1.0

[wikipedia.cpu_intel]
page_url = "https://en.wikipedia.org/wiki/List_of_Intel_processors"

[wikipedia.cpu_amd_ryzen]
page_url = "https://en.wikipedia.org/wiki/List_of_AMD_Ryzen_processors"

[wikipedia.gpu_nvidia]
```

```
page_url = "https://en.wikipedia.org/wiki/List_of_Nvidia_graphics_processing_units"

[wikipedia.gpu_amd]
page_url = "https://en.wikipedia.org/wiki/List_of_AMD_graphics_processing_units"

[huggingface]
endpoint = "https://huggingface.co"
request_delay_sec = 0.2
max_models_total = 2000
max_models_per_team = 200
pipeline_tag = "text-generation"
license_id = "apache-2.0"
inference_provider = "all"
num_parameters = "min:0.3B,max:200B"
teams = ["Qwen", "openai", "meta-llama", "google", "deepseek-ai", "MiniMaxAI", "mistralai", "lgai-exaone", "nvidia"]
```

Hugging Face 요청 구분과 rate limit 대응

Hugging Face는 요청을 Hub APIs / Resolvers(`/resolve/`) / Pages로 구분하고, 각각 다른 레이트리밋 정책을 둔다. canirunai의 모델 수집은 주로 Hub APIs(list_models/model_info)와 일부 Pages 접근에 해당하므로, 기본 throttle과 캐시를 적용해야 한다. [25]

또한 Hub API 문서는 OpenAPI Playgound 및 `.well-known/openapi.json`을 통해 최신 API 레퍼런스를 제공한다고 명시한다. [16]

Wikipedia 변경 추적과 raw_cache 정책

Wikipedia는 page revision 정보를 제공하므로, 수집기 업데이트는 다음 전략을 취한다.

- (권장) `revisions` API로 최신 revision id 확인 후 변경이 없으면 스킵 [26]
- 변경이 있으면 `parse` API로 HTML을 받아 raw_cache 저장 후 파싱 수행 [18]
- 저장된 raw는 향후 파서 개선 시 재처리(replay) 가능하도록 유지

추가로 Wikimedia Core REST API의 HTML endpoint가 “향후 deprecation 예정”이라는 안내가 있으므로, 장기적으로는 MediaWiki REST/Action API 기반으로 수집하는 쪽이 더 안전하다. [27]

Codex를 위한 구현 지침과 수용 기준

이 문서는 Codex 기반 바이브코딩으로 구현될 예정이므로, 구현 우선순위와 “완성 판정 기준(acceptance criteria)”를 명시한다.

구현 우선순위

- Pydantic 스키마부터 고정
- `CpuSpec`, `GpuSpec`, `ModelSpec`, `ScoreReport`를 먼저 구현하고, 모든 파서는 이 스키마를 반드시 만족해야 한다.
- JSON Store/Loader 구현
- atomic write(임시파일→rename), schema_version 체크, canonical_name 인덱싱
- 수집기 구현

- Wikipedia: (1) 페이지 HTML 확보, (2) raw_cache 저장, (3) 파서 호출, (4) canonicalize, (5) merge
- Hugging Face: list_models/model_info 기반(필터 적용) + 필요 시 특정 repo 단건 수집
- OpenAI 파서 구현
- Structured Outputs 기반으로 스키마 강제
- Pydantic validation 실패 시 retry 전략 구현

OpenAI의 Structured Outputs/JSON mode는 "JSON mode는 스키마 보장이 아니라 유효 JSON 보장"이며, JSON mode 사용 시 시스템 메시지에 JSON 출력을 명시해야 한다는 운영상 주의가 문서에 포함되어 있다. [22]
또한 OpenAI는 통합 Responses API를 신형 권장 프리미티브로 설명한다. [28]

- Scoring Engine(LLM estimator) 구현
- 먼저 컨텍스트/VRAM 기반 산정(신뢰도 높음)을 완성
- 그 다음 bandwidth/GFLOPS 기반 tokens/sec 산정(신뢰도 중간)을 추가
- CLI/터미널 출력 구현(마지막)
- CLI는 core에 의존, core는 CLI에 무의존

최소 수용 기준(MVP)

- `update cpu/gpu/model`가 실행되고 JSON이 생성/갱신된다.
- `list/get`가 spec JSON을 기반으로 동작한다.
- `check`가 spec JSON과 사용자 `--memory`만으로 다음 최소 필드를 포함한 JSON을 반환한다.
- `verdict`, `score`, `safe_context_tokens`, `max_supported_context_tokens`, `decode_tokens_per_sec`
- `--output json`은 raw JSON을 그대로 출력한다.
- `--output wide`는 memory/placement/bottleneck/confidence 등을 포함한다(단 "recommendations"는 포함하지 않음).

테스트 전략

- 단위 테스트
- canonical name 정규화(중복 제거, 공백/하이픈/케이스 처리)
- merge 로직(기존 엔트리 업데이트/신규 추가/삭제 정책)
- scoring 하드게이트(IMPOSSIBLE 조건)
- 회귀 테스트
- raw_cache의 "과거 HTML"을 재입력했을 때 파서 출력이 스키마를 만족하는지
- 통합 테스트
- config.toml 샘플로 `update → check`까지 end-to-end

모호/모순 사항 정리(문서 차원 수정)

- `update` resource의 `mode`는 문맥상 `model`로 정정(요구사항 내 자체 모순 해결).
- `update gpu` 설명 중 "cpu 스펙 수집 기능 실행"은 `gpu 스펙 수집 기능 실행`으로 정정.
- "Hugging Face 하위 페이지 순회"는 실제 구현에서 **공식 Hub API 기반 수집 + 필요 시 HTML 보조**로 구현해도 요구사항 목적(웹 기반 수집, source_url 기록, 변동 대응)과 일치한다고 본다. Hub API/필터/확장 속성은 공식 문서에 의해 정의된다. [29]

---

[1] https://en.wikipedia.org/wiki/List_of_Intel_processors
https://en.wikipedia.org/wiki/List_of_Intel_processors

[2] https://en.wikipedia.org/wiki/List_of_AMD_Ryzen_processors
https://en.wikipedia.org/wiki/List_of_AMD_Ryzen_processors

3 13 https://en.wikipedia.org/wiki/List_of_Nvidia_graphics_processing_units
https://en.wikipedia.org/wiki/List_of_Nvidia_graphics_processing_units

4 https://en.wikipedia.org/wiki/List_of_AMD_graphics_processing_units
https://en.wikipedia.org/wiki/List_of_AMD_graphics_processing_units

5 14 https://huggingface.co/docs/hub/model-cards
https://huggingface.co/docs/hub/model-cards

6 https://huggingface.co/docs/hub/repositories-licenses
https://huggingface.co/docs/hub/repositories-licenses

7 10 https://huggingface.co/models
https://huggingface.co/models

8 https://huggingface.co/docs/inference-providers/hub-api
https://huggingface.co/docs/inference-providers/hub-api

9 11 12 15 17 24 https://huggingface.co/docs/huggingface_hub/en/package_reference/hf_api
https://huggingface.co/docs/huggingface_hub/en/package_reference/hf_api

16 29 https://huggingface.co/docs/hub/api
https://huggingface.co/docs/hub/api

18 https://www.mediawiki.org/wiki/API%3AParsing_wikitext
https://www.mediawiki.org/wiki/API%3AParsing_wikitext

19 26 https://www.mediawiki.org/wiki/API%3ARevisions
https://www.mediawiki.org/wiki/API%3ARevisions

20 https://www.mediawiki.org/wiki/API%3AGet_the_contents_of_a_page
https://www.mediawiki.org/wiki/API%3AGet_the_contents_of_a_page

21 https://techcrunch.com/2025/11/10/wikipedia-urges-ai-companies-to-use-its-paid-api-and-stop-scraping/
https://techcrunch.com/2025/11/10/wikipedia-urges-ai-companies-to-use-its-paid-api-and-stop-scraping/

22 https://developers.openai.com/api/docs/guides/structured-outputs/
https://developers.openai.com/api/docs/guides/structured-outputs/

23 28 https://developers.openai.com/api/docs/guides/migrate-to-responses/
https://developers.openai.com/api/docs/guides/migrate-to-responses/

25 https://huggingface.co/docs/hub/rate-limits
https://huggingface.co/docs/hub/rate-limits

27 https://api.wikimedia.org/wiki/Core_REST_API/Reference/Pages/Get_HTML
https://api.wikimedia.org/wiki/Core_REST_API/Reference/Pages/Get_HTML